

SDE III / IV · INTERVIEW PREP

The Machine Coding round.

How we evaluate the way a senior engineer turns requirements into a clean design, working code, and the right patterns — with 90 minutes on the clock.

- WHAT THIS ROUND MEASURES

Production-quality code, in 90 minutes.

We evaluate how you translate a fresh requirement into **clean, maintainable, production-quality code** — with the design instincts of a senior engineer.

Design first.
Then code.

TIME ON THE CLOCK

90 min

From reading the spec to modelling entities, picking patterns, writing the code, adding tests, and demoing it back.

● A TASTE OF THE ROUND

Two example questions.

EXAMPLE 01 FRAMEWORK DESIGN

Logging framework.

Build an in-process logger. Levels, multiple destinations, configurable per logger, safe to call from anywhere in the app.

MUST SUPPORT

- Levels: DEBUG / INFO / WARN / ERROR, with per-logger thresholds
- Multiple sinks — console, file, in-memory — added at runtime (*Strategy*)
- Pluggable formatters: plain, JSON, custom (*Strategy / Factory*)
- Async dispatch with a bounded queue and graceful shutdown (*concurrency*)

EXAMPLE 02 CONCURRENT SYSTEM

Rate limiter.

Build an in-memory rate-limiting service that protects an API. Multiple algorithms, multiple clients, all concurrent.

MUST SUPPORT

- Per-client and per-endpoint quotas, configured at startup
- Pluggable algorithms — token bucket, sliding window (*Strategy*)
- Correct under N concurrent `allow()` calls (*concurrency*)
- Pre-checks composable with auth, IP filters, etc. (*Chain of Responsibility*)

Indicative only. Your actual question will be in this shape and difficulty, but different.

● BASELINE EXPECTATIONS

What 'good' looks like.

01

Working and demo-able.

The code compiles and executes end to end. You can walk us through a live demo of every requirement in the spec.

02

Correct and unit-tested.

Outputs match the spec. Unit tests cover the main paths and the edge cases that matter — the suite is part of the deliverable.

03

Modular and readable.

Clear abstractions, a clean entity model, separation of concerns. A new requirement should be a small, local change — not a rewrite.

MANDATORY TO QUALIFY

Code must **compile, execute and complete** the problem. Prioritise this above everything else.

● WHAT SEPARATES THIS ROUND

The senior bar.

DESIGN PATTERNS

04 · A

Patterns used deliberately.

Reach for the right pattern when it fits — not as decoration. We'll ask why you chose it and what the alternative was.

Factory

Strategy

Dependency Injection

Chain of Responsibility

CONCURRENCY

04 · B

Safe under contention.

Identify shared state. Pick the right primitive — lock, atomic, immutable copy, queue. Walk us through how races are prevented, not just where they aren't.

Shared state

Locks & atomics

Async queues

Isolation boundaries

● BEFORE WE START

Pre-requisites to be ready with.



01 · YOUR MACHINE

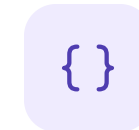
Your own setup, ready to go.

Bring a laptop with an IDE or online editor of your choice. Have it installed, compiling, and authenticated **before the session starts**. You'll be coding live on your own setup.

VS CODE

INTELLIJ

YOUR CHOICE



02 · I/O FORMAT

Flexible — your call.

Input and output format is **completely up to you**. CLI, hard-coded driver, JSON file — whatever lets you demo all the requirements fastest. Spend the time on the design, not the harness.

CLI DRIVER

MAIN()

JSON / FIXTURES

FROM ALL OF US AT HEVO ENGINEERING

All the best.

Questions about the round, your setup, or anything else?
Reach out to your Talent partner. They're your first stop.